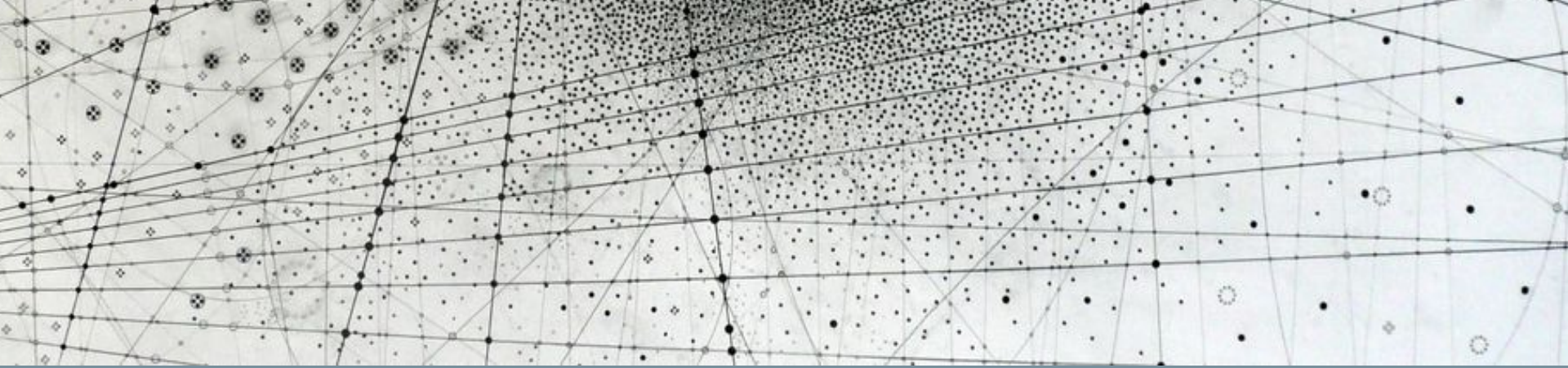




Research Methods in Machine Learning & Physics

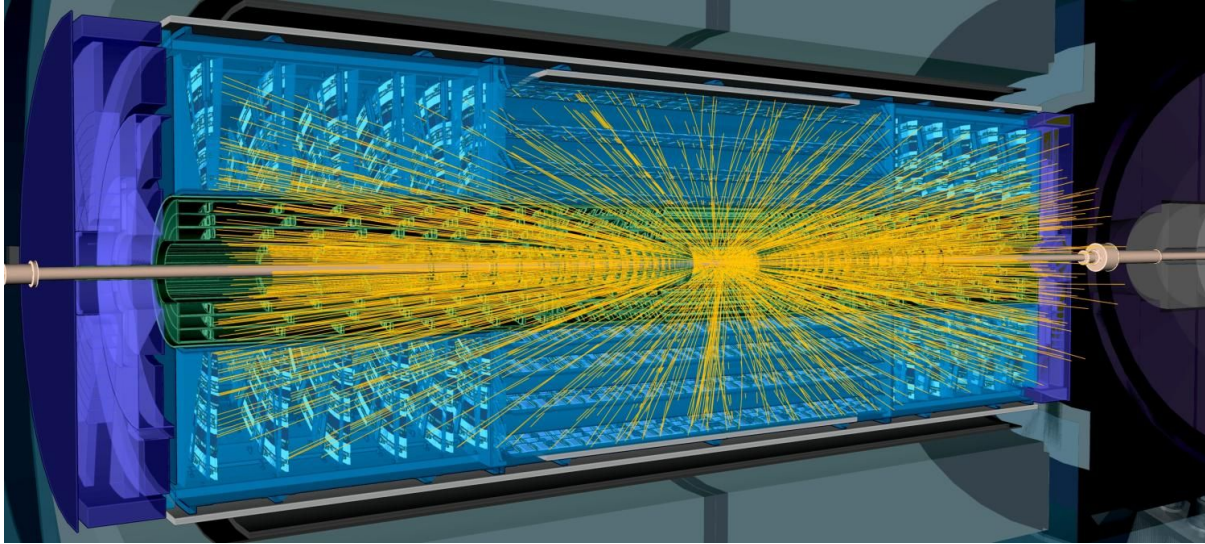
Quick recap

Quick Recap: Hyperparameter Optimization

- A **hyperparameter** is anything you can control that affects the learning process
- Your **validation set** is meant to help you tune your hyperparameters
- Which hyperparameters to tune?
 - As many as is feasible within your time constraints!
 - **Optimizer-related** (e.g. learning rate, choice of optimizer, learning rate scheduler)
 - **Regularizer-related** (e.g. dropout rate, patience, weight decay)
 - **Architecture-related** (e.g. # of layers, layer sizes, activation functions, normalization)
 - **Training-related** (e.g. # of epochs, # of data loader workers, shuffling strategy)
- Batch size:
 - Generally don't tune to improve validation loss
 - Instead, **max out your batch size** in powers of 2 to fit onto your GPU
 - See how different batch sizes affect training speed
- Recommended HPO strategies:
 - **Bayesian Optimization**
 - *Alternate:* Random search OR a coarse grid search, then random search

Dimensionality Reduction

Modern physics detectors capture many (millions!) of dimensions...



...but the important underlying physics is likely much lower in dimensionality.

26 parameters of the Standard Model:

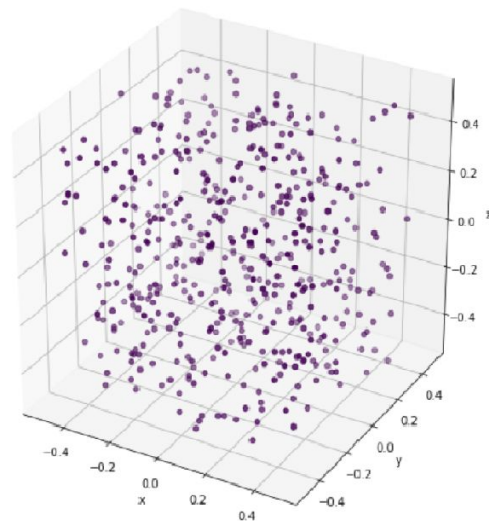
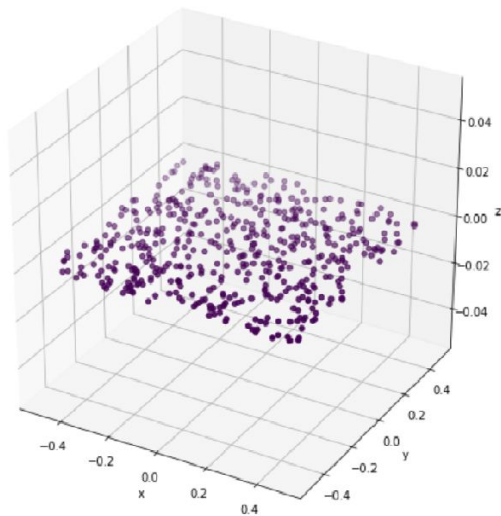
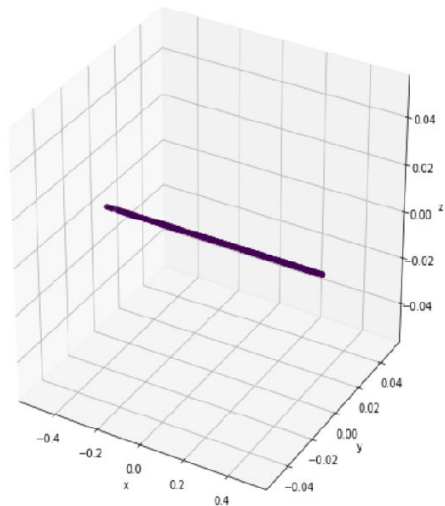
6 quark masses	m_u, m_c, m_t, m_d, m_s and m_b
4 quark mixing angles	$\theta_{12}, \theta_{23}, \theta_{13}$ and δ
6 lepton masses	$m_{\nu_e}, m_{\nu_\mu}, m_{\nu_\tau}, m_e, m_\mu$ and m_τ
4 lepton mixing angles	$\theta'_{12}, \theta'_{23}, \theta'_{13}$ and δ'
3 gauge couplings	$g_1(\alpha), g_2(G_F)$ and g_3
1 Higgs vacuum expectation value	$v(M_Z)$
1 Higgs mass	m_H
1 strong CP violating phase	θ

...but the important underlying physics is likely much lower in dimensionality.

	<i>Planck</i> TT,TE,EE+lowE+lensing	+BAO
$\Omega_b h^2$	0.02237 ± 0.00015	0.02242 ± 0.00014
$\Omega_c h^2$	0.1200 ± 0.0012	0.1193 ± 0.0009
$100 \theta_{\text{MC}}$	1.0409 ± 0.0003	1.0410 ± 0.0003
n_s	0.965 ± 0.004	0.966 ± 0.004
τ	0.054 ± 0.007	0.056 ± 0.007
$\ln(10^{10} \Delta_{\mathcal{R}}^2)$	3.044 ± 0.014	3.047 ± 0.014
h	0.674 ± 0.005	0.677 ± 0.004
σ_8	0.811 ± 0.006	0.810 ± 0.006
Ω_m	0.315 ± 0.007	0.311 ± 0.006
Ω_Λ	0.685 ± 0.007	0.689 ± 0.006

"The Curse of Dimensionality":

The number of data points needed to have a sufficiently populated volume increases dramatically with the number of dimensions.



Dimensionality-reduction techniques can help us **project our complex experimental data into lower-dimensional spaces.**

This can:

- Help visualize our data in 2D or 3D
- Reduce the effects of noise or other redundancies in our data
- Reveal underlying structure

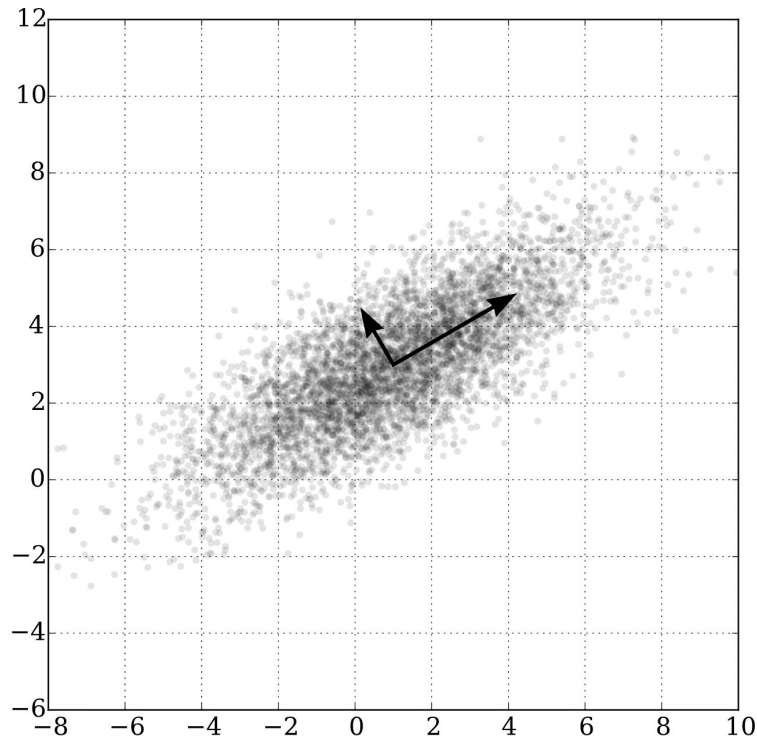
PCA: a **linear** dimensionality-reduction method

PCA = Principal Component Analysis

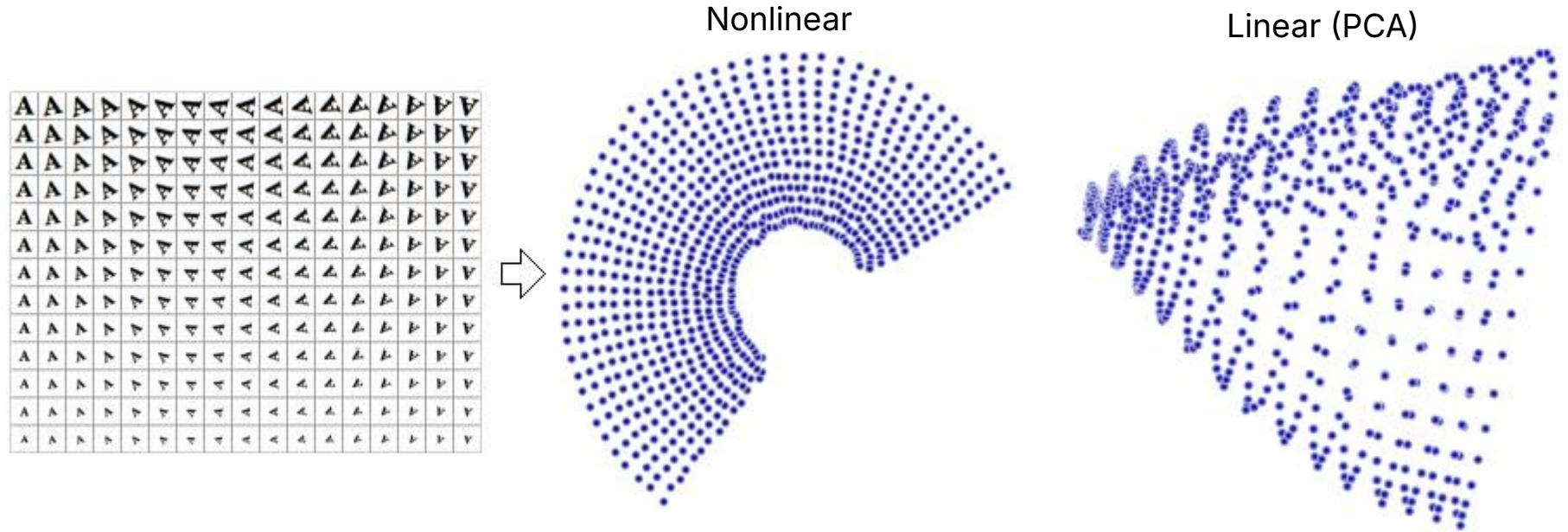
"Can I pick a better coordinate system to describe my data more effectively?"

- Finds decorrelated axes that explain as much of the variation in the data as possible (i.e. eigenvectors of the covariance matrix)
- Axes are ranked by importance:
 - e.g. principal component #1 explains the most variance in the data

Note: only applies linear transformations!



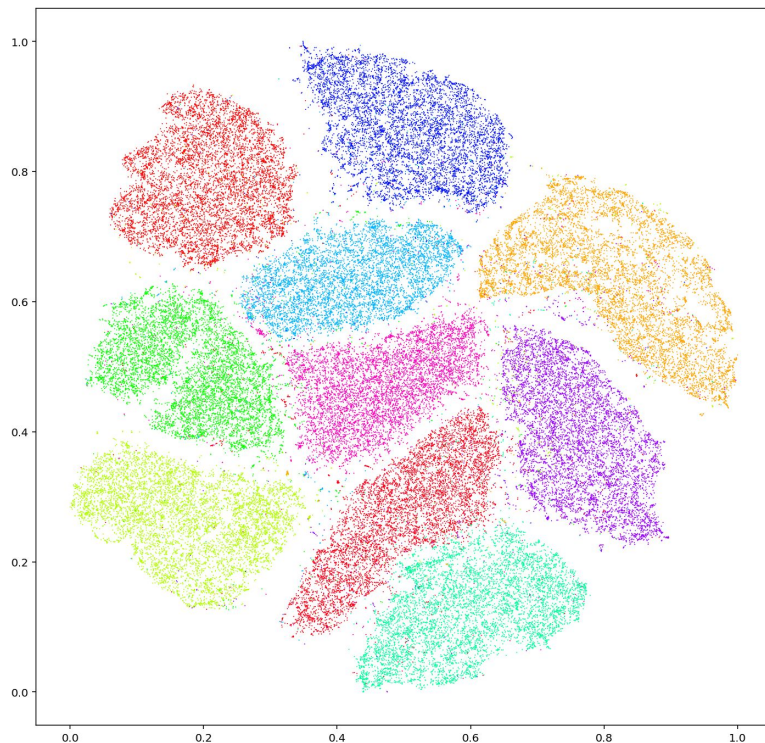
Nonlinear dimensionality reduction is often more expressive.



t-SNE: a **non-linear** dimensionality-reduction method

t-SNE = "t-distributed stochastic neighbor embedding"

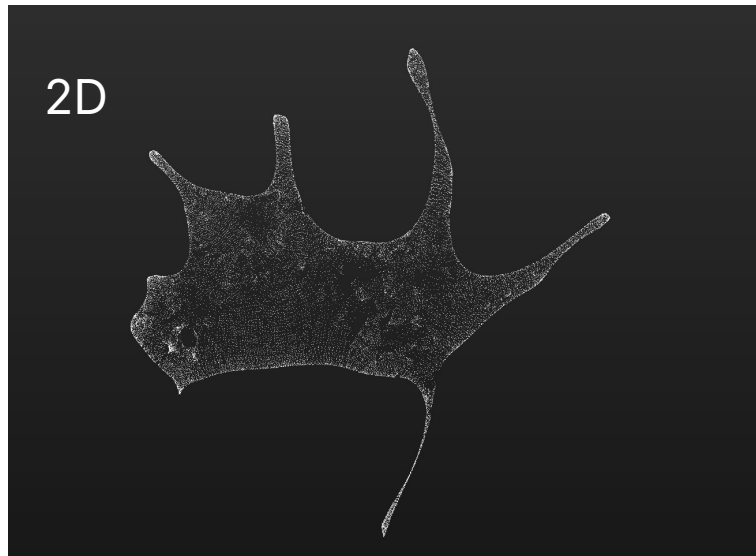
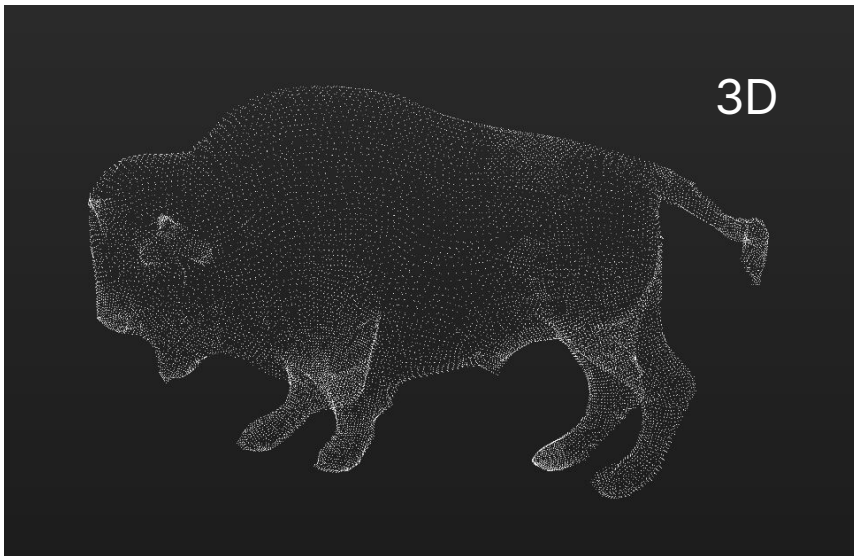
- Computes the probability that pairs of data points in the high-dimensional space are related
- Chooses low-dimensional embeddings that produce a similar distribution by minimizing the KL Divergence
- **Stochastic**: can get different results each time you run



UMAP: a **non-linear** dimensionality-reduction method

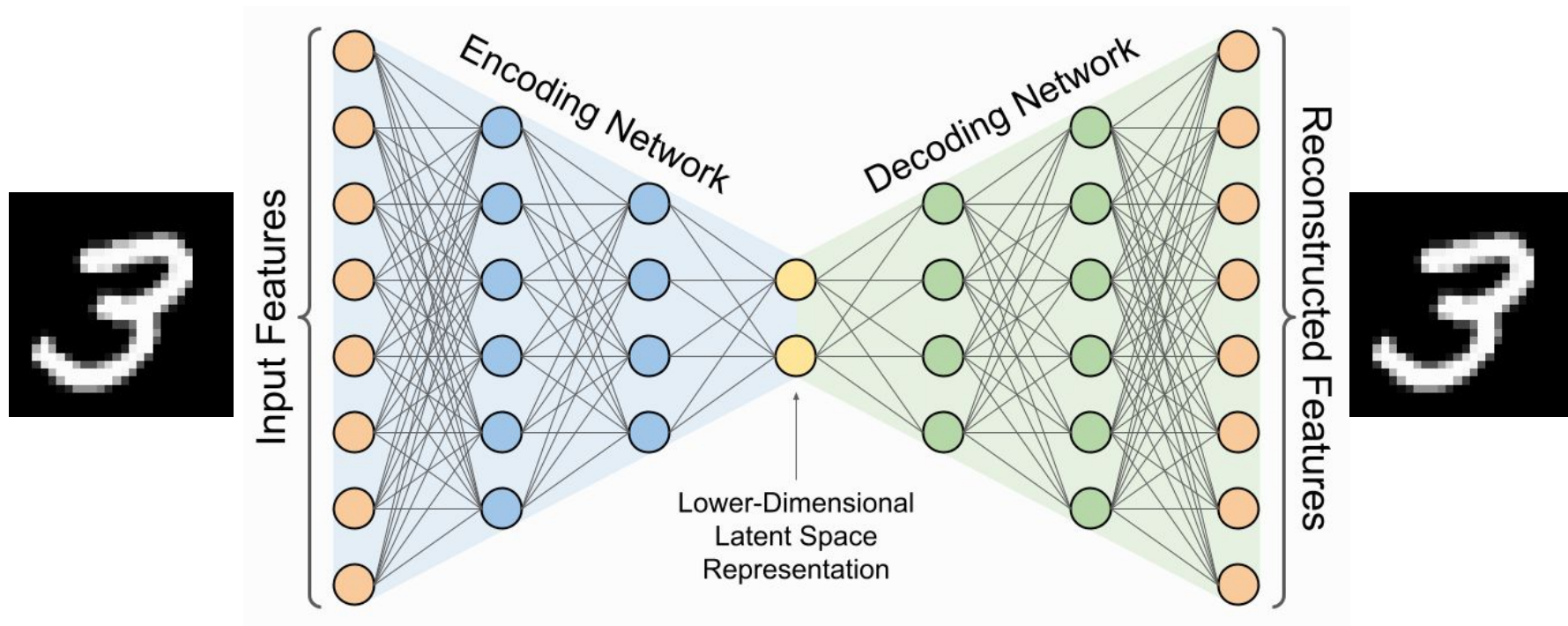
UMAP = “Uniform Manifold Approximation and Projection”

- Assumes the data is uniformly distributed on a locally-connected Riemannian manifold with a locally-constant metric
- Also **stochastic** in nature



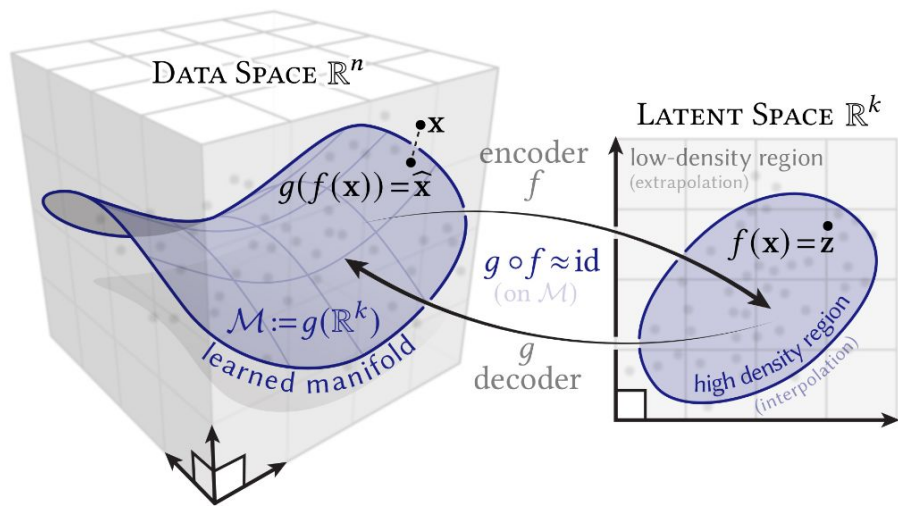
Autoencoders: NN-based dimensionality reduction

Architecture has a signature "bow-tie" shape:

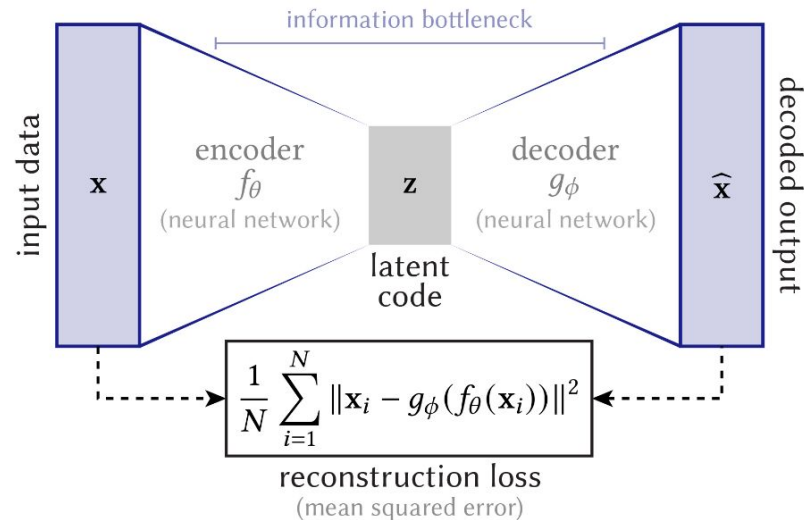


Autoencoders: NN-based dimensionality reduction

What does it represent?



How is it implemented?



In-class work

- **Notebook 2:** playing with dimensionality-reduction techniques

Repository: https://github.com/mariel-petee/phys_805_fall_2025/

PS: A shorter-than-usual Project #1 will be posted by this Friday and due by next weekend